Chapter 8: Classification and Detection

Lecture Handout – The Wolf of Wall Street

Based on lecture notes by Kyunghyun Cho
Lecturer: Lavender Jiang, NYU

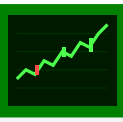
Name: Lecture **Date:** March 9, 2026

Instructions: Follow along during lecture and fill in the blanks. Bring this to class!

Useful values of $\exp(x)$

x	-3	-2	-1	-0.5	0	0.5	1	1.5	2	3	4
$\exp(x)$	0.05	0.14	0.37	0.61	1.00	1.65	2.72	4.48	7.39	20.1	54.6

Also: $\log 2 \approx 0.69$, $\log 3 \approx 1.10$, $\log(1/3) \approx -1.10$.



Scenario: The Wolf of Wall Street

You are building a trading model for a stock market. Each morning you observe a feature vector $a \in \mathbb{R}^d$ for a stock (e.g., yesterday's return, trading volume, price-to-earnings ratio) and must make a decision. Your model outputs logits $u \in \mathbb{R}^K$ which are converted to probabilities.

Two settings:

- **Classification** (mutually exclusive): predict one action from $K = 3$ choices – **Sell**, **Hold**, **Buy** – using softmax.
- **Detection** (non-exclusive): flag which of K market signals are present (e.g., price rising, high trading volume, in the news) – using per-signal sigmoids.

Coming up in lab: You will use what we learn today to build your own trading strategy for a ~~virtual~~ stock market and compete against your classmates!

1980s

1. From Unsupervised to Supervised

Ch 6: Autoenc. predict own input → unsupervised → **Ch 7: K-Means** discover clusters → supervised! → **Ch 8: Classifier** predict ext. label

In Chapters 6–7, the model predicts itself. $x \xrightarrow{\text{enc}} f \xrightarrow{\text{dec}} g \; x'$

In Chapter 8, each input a is paired with an external label denoted $y^\star$.

input

	Autoencoder (Ch 6)	K-Means (Ch 7)	Classifier (Ch 8)
What does it predict?	own input	cluster assignment	labels
Supervision type	self supervised	unsupervised	supervised
Uses external labels?	×	×	✓
Learns features $f(a;\theta)$?	✓	×	✓

Notes:

2. Classification: Softmax

Setup: Each morning you see stock features a and choose one action.

label label

$$\text{features } a \longrightarrow \text{neural net } f(a;\theta) \longrightarrow \text{logits } u \longrightarrow \text{softmax} \longrightarrow p(y|a)$$

param

Classification: y^* is one of K mutually exclusive categories. Buy (A) Sell (B) Hold (C) exp(3) exp(-100)

The softmax function converts logits $u \in \mathbb{R}^K$ to probabilities: 3 -100 200

$$p(y = k \mid a) = \frac{\exp(u_k)}{\sum_j \exp(u_j)}$$

① $p \in [0,1]$

Properties: All outputs are in [0, 1], and they sum to 1.

$\exp(x) \geq 0$

Worked example: $u = (0, 1, 2)$ for Sell / Hold / Buy.

	Sell	**Hold**	**Buy**
u_k	0	1	2
$\exp(u_k)$	exp(0)	e^1	e^2
$p(y = k\|a)$	$\frac{e^0}{Z}$	$\frac{e^1}{Z}$	$\frac{e^2}{e^0+e^1+e^2}$

$Z =$ $e^0 + e^1 + e^2$

Recommendation: Buy

sigma-like

Binary classification: Show that sigmoid is softmax with $K = 2$ and logits $(a, 0)$.
Write the softmax probability for the positive class ($k = 1$):

pos logit

$$p(y = 1 \mid a) = \boxed{\frac{\exp(a)}{\exp(a) + \exp(0)}}$$

Divide numerator and denominator by $\exp(a)$:

$$= \frac{\quad}{\quad + \quad} = \frac{1}{1 + \exp(-a)} = \sigma(a)$$

So sigmoid is a special case of softmax.

$\frac{1}{1+0}$ ∞ $-\infty$ $\frac{1}{\infty}$

$\boxed{\frac{1}{1+\exp(-a)}}$

Σ 6

Notes:

① Cross entropy

② Binary classification

: $CEL = -1 \boxed{\log} \boxed{q_{y^*}}$ $\frac{\boxed{\exp}(u_{y^*})}{\sum_j \exp(u_j)}$

$= -u_{y^*} - (-\log \sum_j \exp(u_j))$

q - predicted distribution

$H(p) = -\sum \boxed{p} \log p$

$\boxed{H(p,q) = -\sum_{i=0}^{i=1} p_i \log q_i}$

3. Cross-Entropy Loss

Binary classification

If the correct action was $y^\star$, the cross-entropy loss is:

$$\mathcal{L} = -\log p(y^\star \mid a) = -u_{y^*} - \left(-\log \sum_j \exp(u_j)\right)$$

$$\cancel{-p_0 \log q_0} - \boxed{p_1 \log q_1}$$

$p_0 = 0$ $p_1 = 1$

$y^* = 1$ correct label.

Intuition: Cross-entropy rewards putting more probability on the correct answer. If $p(y^\star) = 1$, loss $= 0$. If $p(y^\star) \approx 0$, loss $\to \infty$.

Comparison: Using $u = (0, 1, 2)$ with $y^\star = \text{Buy}$:

ex.	**Our model** $u = (0, 1, 2)$	**Know-nothing model** $u = (0, 0, 0)$
$p(\text{Buy})$	_____	_____
$\mathcal{L} = -\log p(\text{Buy})$	_____	_____

Softmax

$$\frac{\exp(u_{y^*})}{\sum_j \exp(u_j)}$$

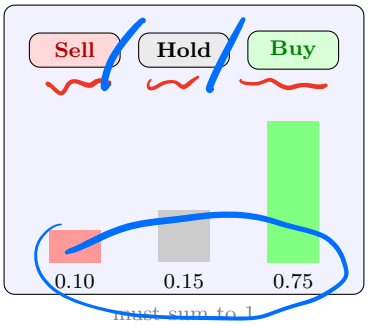
u_1

Notes:

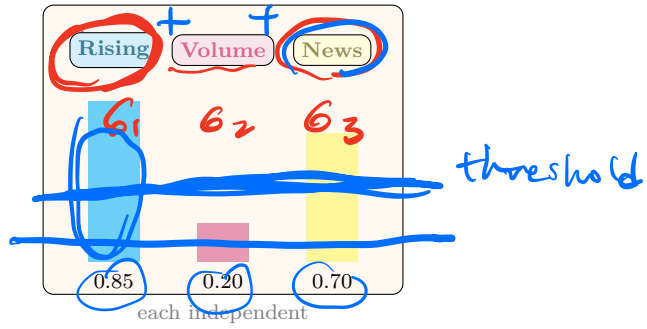
4. Detection: Sigmoid

New problem: A stock can show *multiple* signals at once (price rising AND in the news).

Classification (softmax)



Detection (sigmoids)



Detection differs from classification because categories are non-exclusive.

Each signal uses an independent sigmoid: $\sigma(a) = \frac{1}{1+\exp(-a)}$

Key insight: sigmoid = 2-class softmax with one logit fixed to 0.

	Classification (Softmax)	Detection (Sigmoid)
Pick how many?	exactly one	any subset.
Output constraint	sum to 1	each $\in [0, 1]$
Example		

Problem with detection labels: Annotators reliably spot positives, but often miss things. So $y_k^\star = 0$ doesn't necessarily mean the object is absent.

The standard detection loss treats positives and negatives equally:

$$\mathcal{L} = \sum_{k=1}^{K} \left[y_k^\star \log \sigma(l_k) + (1 - y_k^\star) \log(1 - \sigma(l_k)) \right]$$

Fix – reweigh negatives: Since we don't fully trust $y_k^\star = 0$, we multiply the negative term by a weight $\omega_k < 1$:

$$\mathcal{L} = \sum_{k=1}^{K} \left[y_k^\star \log \sigma(l_k) + \omega_k (1 - y_k^\star) \log(1 - \sigma(l_k)) \right]$$

Since $\omega_k < 1$, this downweight the contribution of negatives.

In practice: use one ω for all k (e.g., $\omega = 0.1$).

Notes:

5. From Detection to Counting

Detection is binary: is a signal present or absent?

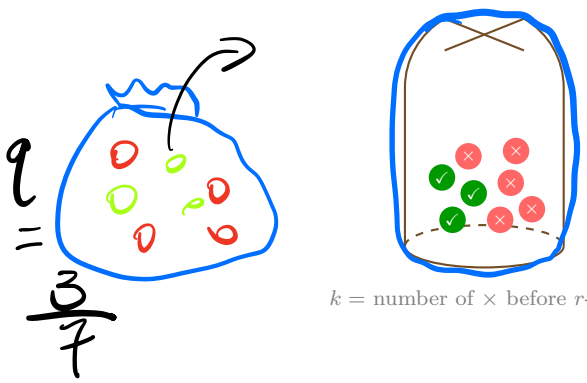
Counting generalizes this: how many instances are there

Trading example: not just "is the price rising?" but "how many buy signals fired today?"

The count is uncertain, so we model it with a **negative binomial distribution**:

$$p(k; r, q) = \boxed{\binom{k+r-1}{k}}(1-q)^k q^r$$

Parameters: $r > 0$ (number of successes), $q \in [0, 1]$ (success probability), $k = 0, 1, 2, \ldots$ (count of failures).



Draw until r successes (✓)

Count the failures (×) $= k$

Each draw: $P(✓) = q$

Problem: A neural net outputs unconstrained values in $\mathbb{R}$, but $\boxed{r > 0}$ (#) and $q \in [0, 1]$.

Fix – change of variables:

$c = \log r$ (so $r =$ exp(c)) $\qquad \boxed{l = \log \frac{q}{1-q}}$ (the logits of q)

Now $c \in \mathbb{R}$ and $l \in \mathbb{R}$ – no constraints! A neural net outputs (c, l) directly.

Derive q from l: Start from $\exp(l) = \frac{q}{1-q}$ and solve for q:

$$(\exp(l))^{-1} = \left(\frac{q}{1-q}\right)^{-1} \iff \boxed{\frac{1}{q} - 1} = \exp(-l)$$

$$\exp(-l) \qquad \frac{1-q}{q} \qquad \left(\frac{1}{q}\right)^{-1} = (\exp(-l) + 1)^{-1}$$

$$q = \frac{1}{\exp(-l) + 1} = \sigma(l)$$

The probability q is the sigmoid of l. (Same function as detection!)

Why is this nice?

- Derivatives w.r.t. l are well-defined (no constrained optimization)
- $\sigma(l)$ has a clean derivative: $\sigma'(l) = \sigma(l)(1 - \sigma(l))$
- The neural net outputs 2 unconstrained numbers per object type

exp(c) σ(ℓ) # of failures

Mean and variance of the negative binomial (k = number of failures):

$$\boxed{\mathbb{E}[k]} = r\frac{1-q}{q} = \frac{\exp(c)\,1-\sigma(\ell)}{\sigma(\ell)} \qquad \mathbb{V}[k] = \frac{1}{q}\boxed{\mathbb{E}[k]} = \frac{r(1-q)}{q^2}$$

Quick check: If $q \to 1$ (almost always succeed), expected failures $\to$ 0.
If $q \to 0$ (almost always fail), expected failures $\to$ ∞.

Notes:

6. KL Divergence

Minimizing cross-entropy = minimizing KL divergence between $\boxed{p^\star}$ gt and $\boxed{p_\theta}$.

Proof: Start from the KL divergence definition and show it equals cross-entropy + constant.

$$\mathrm{KL}(p^\star \| p_\theta) = -\sum_{k=1}^{K} p^\star(k) \log \boxed{\frac{p_\theta(k)}{p^\star(k)}}$$

$p_\theta(k) = p^*(k)$ 1 $\log 1 = 0$

$\log \frac{a}{b} = \log a - \log b$

$$= -\sum_{k} p^\star(k) \log p_\theta(k) + \sum p^*(k) \log p^*(k)$$

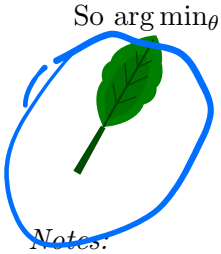
$$= \underbrace{\qquad\qquad}_{\text{cross-entropy}} + \underbrace{\qquad\qquad}_{\text{const. w.r.t. } \theta}$$

CE

$\boxed{\text{entropy of } p^*}$

The second term is entropy of $p^\star$, which doesn't depend on θ.

So $\arg\min_\theta \mathrm{KL}(p^\star \| p_\theta) = \arg\min_\theta$ CEL.



Notes:

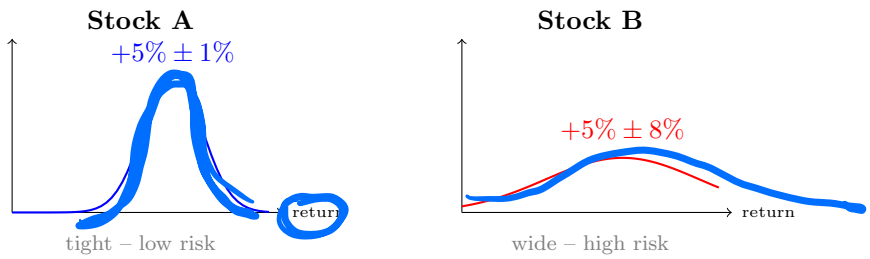
7. Looking Ahead

Classification tells us direction (up or down), but a trader also needs:

- The magnitude of the expected move (how many %?)
- The uncertainty around that prediction (how sure?)

0.001 %

Example: Two stocks both classified "90% Buy" – but very different risk:



Same direction, same confidence – but very different risk!

Chapter 9 (Regression & Uncertainty) gives us tools to tell them apart:

_____

Notes:

Bonus Sections

The following sections cover additional topics.
We may not have time to cover them all in lecture – read ahead if you're curious!

B1. Evaluation: Precision & Recall

Your "Buy" detector flags stocks it thinks will go up. You test on 20 stocks:

	Actually Up	**Actually Down**	
Predicted Up	**TP = 8**	**FP = 2**	10 flagged
Predicted Down	**FN = 4**	**TN = 6**	10 not flagged
	12 truly up	8 truly down	

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + ____} = _____ \qquad \text{Recall} = \frac{\text{TP}}{\text{TP} + ____} = _____$$

In trader's words:

- Precision: "Of all stocks I _____, what % actually _____?"
- Recall: "Of all stocks that _____, what % did I _____?"

Threshold tradeoff: We predict "Buy" when $p(\text{up}|a) > \tau$.

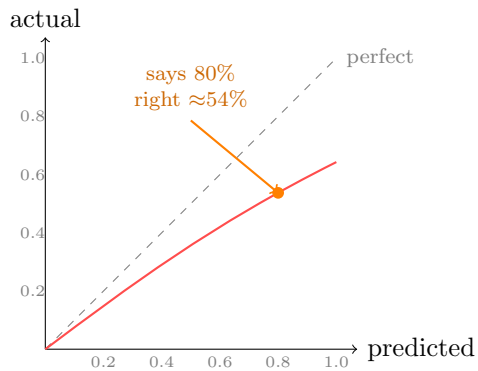
- Lower $\tau \Rightarrow$ flag more $\Rightarrow$ recall _____, precision _____ (aggressive)
- Raise $\tau \Rightarrow$ flag fewer $\Rightarrow$ precision _____, recall _____ (conservative)

A risk-averse trader prefers high _____ and would _____ the threshold.

Summarizing the tradeoff: As we sweep τ from 1 to 0, we trace a _____ curve.
The area under this curve is called _____ (AP).
With K categories: mAP = _____ (mean average precision)

Notes:

B2. Calibration & Temperature Scaling



A **calibrated** model: when it says $p = 0.8$, the outcome occurs _____% of the time.

Why it matters: An overconfident model makes traders _____.

Temperature scaling – a post-hoc fix:

$$p(y = k \mid a; \theta, T) = \frac{\exp(u_k/T)}{\sum_j \exp(u_j/T)}$$

Example: $u = (0, 1, 2)$. Compute at $T = 1$ (already done!) and $T = 2$.
Hint for $T = 2$: $u/T = (0,\ 0.5,\ 1)$. Use the lookup table!

	$T = 1$ (standard)	$T = 2$ (softer)
p(Sell)	_____	_____
p(Hold)	_____	_____
p(Buy)	_____	_____

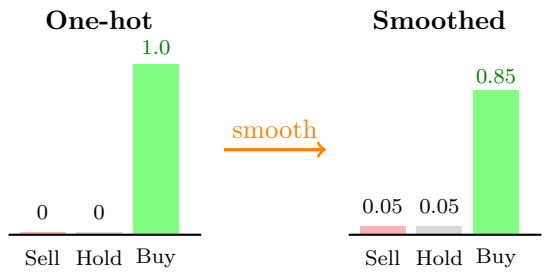
	$T \to 0^+$	$T = 1$	$T \to \infty$
Distribution	_____	_____	_____
Confidence	_____	_____	_____
Ch 7 analog	_____	–	_____

Notes:

B3. Label Smoothing

Problem: One-hot labels drive $p(y^\star|a) \to$ _____, needing $u_{y^\star} \to$ _____.
The model becomes absolutely certain about every label – even noisy ones!

Label smoothing makes targets slightly soft – like adding kale to your diet:



$$\tilde{p}(y = k) = \begin{cases} 1 - \alpha + \alpha/K, & k = y^\star \\ \alpha/K, & \text{otherwise} \end{cases}$$

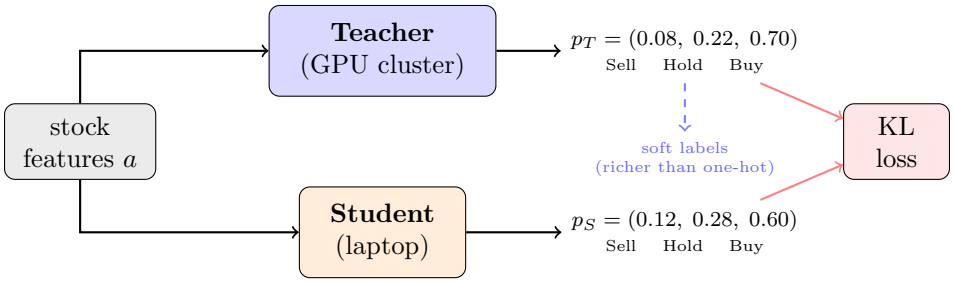
Example: $\alpha = 0.2$, $K = 4$ classes, $y^\star =$ Buy:

$\tilde{p}(\text{Sell}) =$ _____, $\tilde{p}(\text{Hold}) =$ _____, $\tilde{p}(\text{Buy}) =$ _____, $\tilde{p}(\text{Short}) =$ _____

Label smoothing acts as _____ that prevents _____.
In trading: a day labeled "Buy" might have been borderline. Smoothing acknowledges this.

Notes:

B4. Knowledge Distillation



Teacher: _____ model on _____. **Student:** _____ model on _____.

Loss: $\mathcal{L}_{\text{distill}} =$ _____

Why soft labels beat one-hot:

	One-hot label	**Teacher's soft label**
Stock X	$(0, 0, 1)$	$(0.08,\ 0.22,\ 0.70)$
Stock Y	$(0, 0, 1)$	$(0.03,\ 0.04,\ 0.93)$
What does student learn?	_____	_____

Cost analysis: Teacher: 1000 stocks/hr on GPU ($5/hr). Fund evaluates 2000 stocks/day, 250 trading days/yr. Student runs free on laptop. Distillation takes 40 GPU-hours.

- Teacher runs _____ hrs/day. Annual cost: _____ $\times$ \$5 $\times$ 250 = _____
- Annual student cost: _____
- Distillation: 40 $\times$ \$5 = _____
- Payback: _____

Same idea at LLM scale: distillation is how billion-parameter models run on your phone.

Notes:

Appendix: Mean and Variance of the Negative Binomial

The negative binomial counts failures k before r successes, with $P(\text{success}) = q$.

Mean. Each "round" ends with a success. In each round, the expected number of failures before a success is:

$$\mathbb{E}[\text{failures per round}] = \frac{1-q}{q}$$

(This is the mean of a geometric distribution with success probability q.)

Since we need r successes (i.e., r independent rounds):

$$\mathbb{E}[k] = r \cdot \frac{\qquad\qquad}{\qquad\qquad}$$

Variance. The variance of a single geometric round is $\frac{1-q}{q^2}$.

Since the r rounds are independent, variances add:

$$\mathbb{V}[k] = r \cdot \frac{\qquad\qquad}{\qquad\qquad} = \frac{1}{q} \cdot \mathbb{E}[k]$$

Note: $\mathbb{V}[k] > \mathbb{E}[k]$ whenever $q < 1$. This is called _____ (variance exceeds the mean), which is why the negative binomial is useful for modeling "bursty" count data.

In terms of the neural net outputs ($c = \log r$, $l =$ logit of q):

$$\mathbb{E}[k] = \exp(c) \cdot \frac{1-\sigma(l)}{\sigma(l)} \qquad \mathbb{V}[k] = \frac{\exp(c) \cdot (1-\sigma(l))}{\sigma(l)^2}$$

End of handout. See you in lecture!